Chapitre 6 : Composants

> **Objectifs** : découper une interface en composants réutilisables, faire circuler les données (props descendantes, événements montants), utiliser les slots, composer une application. *Composition API (`<script setup>`).*

> **Comment lire les exemples** : chaque exemple indique le **nom du fichier** en commentaire de première ligne. `<!-- → ... -->` et `// →` donnent **ce qui s'affiche ou la valeur obtenue**. Pour tester, créez les fichiers dans `src/components/` d'un projet `create-vue`, puis utilisez le composant parent dans `App.vue`.

---

1. Création et enregistrement d'un composant

1.1 Pourquoi des composants ?

Un **composant** est une brique autonome : sa structure (template), sa logique (script) et son style. Une application est un **arbre de composants**.

```
App
├─ EnTete
├─ ListeTaches
│   ├─ ItemTache
│   ├─ ItemTache
│   └─ ItemTache
└─ PiedDePage
```

Avantages : **réutilisation** (écrit une fois, utilisé partout), **lisibilité** (fichiers courts), **testabilité**, travail en équipe.

1.2 Un composant = un fichier `.vue`

```
<!-- src/components/BoutonCompteur.vue -->
<script setup>
import { ref } from "vue";

const n = ref(0);    // état PROPRE à chaque instance du composant
</script>

<template>
  <button @click="n++">Cliqué {{ n }} fois</button>
</template>
```

1.3 Utilisation

Avec `<script setup>`, un composant **importé** est immédiatement utilisable : il n'y a pas d'enregistrement à faire.

```
<!-- src/App.vue -->
<script setup>
import BoutonCompteur from "./components/BoutonCompteur.vue";   // import =
disponible dans le template
</script>

<template>
  <BoutonCompteur />
  <BoutonCompteur />
  <BoutonCompteur />
  <!-- → trois boutons, tous affichant « Cliqué 0 fois » au départ
       → un clic sur le 2e : ce bouton affiche « Cliqué 1 fois » ; les deux autres
restent à 0
         (chaque instance a son propre état : n est recréé à chaque utilisation) -
->
</template>
```

Conventions de nommage :

- fichiers et noms de composants en **PascalCase** (`BoutonCompteur.vue`) ;
- dans le template, on écrit `<BoutonCompteur />` (la forme kebab-case `<bouton-compteur>` est aussi acceptée) ;
- un nom de composant doit être **composé de plusieurs mots** : cela évite les conflits avec les balises HTML (`Bouton.vue` pourrait un jour entrer en conflit, `BoutonCompteur.vue` non) ;
- convention : préfixe `App` pour les composants uniques de l'application (`AppEnTete`), `Base` pour les composants de base très génériques (`BaseBouton`).

---

2. Props : passer des données du parent à l'enfant

2.1 Déclarer et utiliser des props

Une **prop** est une donnée que le **parent** donne à l'**enfant**, comme un paramètre de fonction. L'enfant la **déclare** avec `defineProps`.

```
<!-- src/components/CarteProfil.vue -->
<script setup>
// defineProps déclare les props acceptées et leur type (macro du compilateur :
pas d'import)
const props = defineProps({
  nom: String,
  age: Number
});

console.log(props.nom);   // → "Ali"   (dans le script : on passe par l'objet
props)
</script>

<template>
```

```
  <!-- Dans le template : on utilise directement nom et age -->
  <div class="carte">
    <h2>{{ nom }}</h2>
    <p>{{ age }} ans</p>
  </div>
</template>
```

```
<!-- Parent : App.vue -->
<script setup>
import { ref } from "vue";
import CarteProfil from "./components/CarteProfil.vue";

const personnes = ref([
  { id: 1, nom: "Ali", age: 20 },
  { id: 2, nom: "Ines", age: 22 }
]);
</script>

<template>
  <!-- Texte fixe : nom="Ali" ; toute autre valeur (nombre, variable, objet) :
avec « : » -->
  <CarteProfil nom="Ali" :age="20" />
  <!-- → <div class="carte"><h2>Ali</h2><p>20 ans</p></div> -->

  <!-- Une carte par personne, grâce à v-for -->
  <CarteProfil v-for="p in personnes" :key="p.id" :nom="p.nom" :age="p.age" />
  <!-- → deux cartes : « Ali — 20 ans » puis « Ines — 22 ans » -->

  <!-- ⚠ Oubli des deux-points sur une valeur numérique : on passe la CHAÎNE "20"
-->
  <CarteProfil nom="Salah" age="20" />
  <!-- → s'affiche quand même « 20 ans », mais la console avertit :
         [Vue warn]: Invalid prop: type check failed for prop "age". Expected
Number with value 20, got String with value "20". -->
</template>
```

À retenir :

- un texte fixe s'écrit `nom="Ali"` ; **toute autre valeur** (nombre, booléen, variable, objet, tableau) s'écrit avec `:` (`:age="20"`) ;
- dans le **template**, on utilise `nom` directement ; dans le **script**, on passe par `props.nom` ;
- le nom d'une prop en JavaScript est en camelCase (`nomComplet`) ; dans le template parent, on peut l'écrire en kebab-case (`nom-complet="…"`) ou en camelCase ;
- `defineProps` est une **macro du compilateur** (chapitre 3, §6.3) : elle n'est pas importée.

```
<!-- Les noms composés : camelCase déclaré, kebab-case utilisé -->
<script setup>
defineProps({ nomComplet: String });     // déclaration en camelCase
</script>
```

```
<!-- Parent : <CarteProfil nom-complet="Ali Ben Salah" /> ou <CarteProfil
:nomComplet="..."/> → les deux fonctionnent -->
```

2.2 Validation des props

On peut décrire précisément chaque prop : type, obligatoire ou non, valeur par défaut, règle de validation.

```
<!-- src/components/CarteNote.vue -->
<script setup>
const props = defineProps({
  // Obligatoire : un avertissement apparaît si le parent l'oublie
  titre: { type: String, required: true },

  // Valeur par défaut + validateur personnalisé (retourne true si valide)
  note: {
    type: Number,
    default: 0,
    validator: (v) => v >= 0 && v <= 20
  },

  // Pour un tableau ou un objet, la valeur par défaut doit être une FONCTION qui
le crée
  tags: {
    type: Array,
    default: () => []          // chaque instance reçoit son propre tableau
  },

  // Valeur limitée à une liste
  statut: {
    type: String,
    default: "actif",
    validator: (v) => ["actif", "inactif"].includes(v)
  },

  etudiant: Object,
  rappel: Function,
  visible: { type: Boolean, default: false },

  // Plusieurs types acceptés
  identifiant: [String, Number]
});
</script>

<template>
  <div>{{ titre }} : {{ note }}/20 ({{ statut }})</div>
</template>
```

```
<!-- Utilisations et avertissements en mode développement -->
<CarteNote titre="Maths" :note="15" />
```

```
<!-- → Maths : 15/20 (actif)     (statut prend sa valeur par défaut « actif ») -->

<CarteNote :note="15" />
<!-- → affiche « : 15/20 (actif) » ET la console avertit :
       [Vue warn]: Missing required prop: "titre" -->

<CarteNote titre="Maths" :note="25" />
<!-- → [Vue warn]: Invalid prop: custom validator check failed for prop "note". --
>

<CarteNote titre="Maths" statut="archive" />
<!-- → [Vue warn]: Invalid prop: custom validator check failed for prop "statut".
-->
```

Types acceptés : `String`, `Number`, `Boolean`, `Array`, `Object`, `Function`, `Date`, `Symbol`, ou une classe. Les erreurs de validation sont de simples **avertissements console** en développement ; elles n'empêchent pas l'affichage et disparaissent en production.

2.3 Booléens

Une prop déclarée `Boolean` peut s'écrire **sans valeur** : sa simple présence signifie `true`.

```
<!-- src/components/BoutonAction.vue -->
<script setup>
defineProps({ desactive: Boolean });
</script>
<template>
  <button :disabled="desactive"><slot /></button>
</template>
```

```
<BoutonAction desactive>Envoyer</BoutonAction>
<!-- → bouton grisé : équivaut à :desactive="true" -->
<BoutonAction :desactive="false">Envoyer</BoutonAction>
<!-- → bouton actif -->
<BoutonAction>Envoyer</BoutonAction>
<!-- → bouton actif (une prop Boolean absente vaut false) -->
```

2.4 Le flux de données est **unidirectionnel**

Les données **descendent** du parent vers l'enfant. **Un composant ne doit jamais modifier une de ses props** : elles sont en lecture seule.

```
<!-- src/components/CompteurMauvais.vue -->
<script setup>
const props = defineProps({ compteur: Number });
```

```
function incrementer() {
  props.compteur++;        // ❌ INTERDIT
}
// → au clic : la console affiche
//   [Vue warn]: Set operation on key "compteur" failed: target is readonly.
//   et la valeur ne change PAS.
</script>
<template><button @click="incrementer">{{ compteur }}</button></template>
```

Pourquoi ? Si l'enfant pouvait modifier ce que le parent lui donne, **deux composants modifieraient la même donnée** et il deviendrait impossible de savoir qui l'a changée. Le sens unique rend le flux **prévisible** : l'état appartient à un seul endroit.

Deux cas courants et leurs solutions :

```
<script setup>
import { ref, computed } from "vue";

const props = defineProps({ valeurInitiale: Number, nom: String });

// Solution 1 : la prop sert de valeur INITIALE à un état LOCAL, que l'enfant peut
modifier
const compteur = ref(props.valeurInitiale);
// Attention : si le parent change plus tard valeurInitiale, compteur ne suit PAS
(copie à la création)

// Solution 2 : calculer une valeur DÉRIVÉE de la prop (reste synchronisée avec
elle)
const nomMajuscules = computed(() => props.nom.toUpperCase());
// → avec nom = "Ali" : « ALI » ; si le parent passe nom = "Ines" plus tard : «
INES » automatiquement
</script>
```

> Pour **demander un changement** au parent, l'enfant **émet un événement** (section 3).

2.5 Objets et tableaux en props

Les objets et tableaux sont passés par **référence** (chapitre 1) : l'enfant reçoit le **même objet** que le parent.

```
// Dans l'enfant
const props = defineProps({ etudiant: Object });
props.etudiant.nom = "Autre";     // ⚠ AUCUN avertissement : on ne remplace pas
la prop, on modifie son contenu
// → l'objet du PARENT est modifié aussi (même référence) : le parent affiche «
Autre »
```

C'est une **mauvaise pratique** : l'état est modifié dans un composant qui n'en est pas propriétaire. Privilégiez l'émission d'événements.

2.6 Déstructuration réactive des props

Depuis Vue 3.5, on peut déstructurer `defineProps` en gardant la réactivité (le compilateur réécrit les accès en `props.xxx`) :

```
const { nom, age = 18 } = defineProps(["nom", "age"]);
// age vaut 18 si le parent ne le fournit pas
```

Nous conservons la forme `props.xxx` dans ce cours, plus explicite.

---

3. Événements personnalisés : de l'enfant vers le parent

3.1 `defineEmits` et `emit`

L'enfant **émet** un événement, comme un clic, que le parent **écoute** avec `@`. L'événement peut transporter des données.

```
<!-- src/components/ItemTache.vue -->
<script setup>
const props = defineProps({ tache: Object });

// On déclare les événements que ce composant peut émettre
const emit = defineEmits(["supprimer", "basculer"]);
</script>

<template>
  <li :class="{ fait: tache.fait }">
    <!-- Cocher la case émet « basculer » en transmettant l'identifiant de la
tâche -->
    <input type="checkbox" :checked="tache.fait" @change="emit('basculer',
tache.id)">
    {{ tache.texte }}
    <!-- Le bouton émet « supprimer » avec l'identifiant -->
    <button @click="emit('supprimer', tache.id)">×</button>
  </li>
</template>

<style scoped>
.fait { text-decoration: line-through; color: gray; }
</style>
```

```
<!-- Parent : App.vue -->
<script setup>
import { ref } from "vue";
import ItemTache from "./components/ItemTache.vue";

// L'ÉTAT appartient au parent
```

```
const taches = ref([
  { id: 1, texte: "Apprendre Vue", fait: false },
  { id: 2, texte: "Faire les exercices", fait: false }
]);

// Le parent réagit à l'événement : c'est LUI qui modifie ses données
function supprimer(id) {
  taches.value = taches.value.filter(t => t.id !== id);
}
function basculer(id) {
  const t = taches.value.find(t => t.id === id);
  t.fait = !t.fait;
}
</script>

<template>
  <ul>
    <!-- @supprimer écoute l'événement émis par l'enfant ; la valeur transmise
(id) est passée à la fonction -->
    <ItemTache v-for="t in taches" :key="t.id" :tache="t"
               @supprimer="supprimer" @basculer="basculer" />
  </ul>
  <!-- Au départ → deux lignes : « Apprendre Vue » et « Faire les exercices »
(cases décochées)
       → on coche la 1re : l'enfant émet basculer(1) ; le parent modifie la tâche
; la ligne devient barrée et grise
       → clic sur × de la 2e : l'enfant émet supprimer(2) ; le parent filtre sa
liste ; la ligne disparaît -->
</template>
```

Résumé du principe : **props down, events up** (données vers le bas, événements vers le haut).

```
        Parent  (détient l'état, le modifie)
   props ↓     ↑ emit
        Enfant  (affiche, signale ce que fait l'utilisateur)
```

L'enfant **ne modifie pas** les données : il **informe** le parent, qui décide.

3.2 Validation des événements

On peut passer un objet pour valider les arguments transmis :

```
const emit = defineEmits({
  // Fonction de validation : retourne true si les arguments sont valides
  ajouter: (payload) => typeof payload === "string" && payload.length > 0,
  annuler: null           // null : aucun contrôle
});

emit("ajouter", "Vue");     // → ok
```

```
emit("ajouter", "");        // → [Vue warn]: Invalid event arguments: event
validation failed for event "ajouter".
```

3.3 Convention de nommage

- On peut émettre en camelCase (`emit("changerNom")`) ; le parent écoute en kebab-case (`@changer-nom`) ou camelCase : **les deux fonctionnent**. Choisissez une convention et gardez-la.
- Nommez l'événement d'après **ce qui s'est passé** (`supprimer`, `valider`, `selection-changee`), pas d'après ce que le parent doit faire.
- Les événements ne **remontent pas** (pas de propagation comme un clic natif) : seul le **parent direct** les reçoit. Pour atteindre un ancêtre plus haut, chaque niveau relaie l'événement, ou on utilise un store (chapitre 9).

3.4 `v-model` sur un composant

Un composant peut supporter `v-model` grâce à `defineModel` (Vue 3.4+). C'est la **liaison bidirectionnelle** entre un état du parent et un état de l'enfant.

```
<!-- src/components/ChampTexte.vue -->
<script setup>
// defineModel retourne un ref synchronisé avec la variable liée par v-model dans
le parent
const modele = defineModel();
</script>

<template>
  <!-- v-model sur l'input lie le champ à « modele » : taper met à jour le parent
-->
  <input v-model="modele">
</template>
```

```
<!-- Parent : App.vue -->
<script setup>
import { ref } from "vue";
import ChampTexte from "./components/ChampTexte.vue";

const nom = ref("Ali");
</script>

<template>
  <ChampTexte v-model="nom" />
  <p>{{ nom }}</p>
  <!-- → au départ : le champ contient « Ali » et le paragraphe affiche « Ali »
       → on remplace par « Ines » dans le champ : le paragraphe affiche « Ines »
immédiatement -->

  <button @click="nom = 'Salah'">Imposer Salah</button>
  <!-- → clic : le champ ET le paragraphe passent à « Salah » (sens parent →
```

```
  enfant aussi) -->
  </template>
```

`defineModel` remplace l'ancien schéma manuel : une prop `modelValue` + un événement `update:modelValue`. On peut aussi avoir **plusieurs modèles nommés** :

```
<!-- FormulaireNom.vue -->
<script setup>
const prenom = defineModel("prenom");
const nom = defineModel("nom");
</script>
<template>
  <input v-model="prenom" placeholder="Prénom">
  <input v-model="nom" placeholder="Nom">
</template>
<!-- Parent : <FormulaireNom v-model:prenom="p" v-model:nom="n" /> -->
```

---

4. Slots : injecter du contenu dans un composant

Un **slot** est un **emplacement** dans le template de l'enfant que le **parent remplit** avec du contenu (texte, HTML, autres composants). Les props transmettent des **données** ; les slots transmettent du **contenu d'interface**.

4.1 Slot par défaut

```
<!-- src/components/CarteBase.vue -->
<template>
  <div class="carte">
    <!-- Le contenu fourni par le parent sera inséré ici ; le texte entre les
balises est le contenu par défaut -->
    <slot>Contenu par défaut</slot>
  </div>
</template>

<style scoped>
.carte { border: 1px solid #ccc; border-radius: 8px; padding: 1rem; }
</style>
```

```
<!-- Parent -->
<CarteBase>
  <h2>Titre</h2>
  <p>Contenu libre fourni par le parent</p>
</CarteBase>
<!-- → <div class="carte"><h2>Titre</h2><p>Contenu libre fourni par le parent</p>
</div> -->
```

```
<CarteBase />
<!-- → <div class="carte">Contenu par défaut</div>    (aucun contenu fourni : le
défaut s'affiche) -->
```

4.2 Slots nommés

Quand un composant a **plusieurs emplacements**, on les distingue par un nom.

```
<!-- src/components/Modale.vue -->
<template>
  <div class="modale">
    <header><slot name="titre">Sans titre</slot></header>   <!-- emplacement nommé
« titre » -->
    <main><slot /></main>                                    <!-- slot par défaut
(sans nom) -->
    <footer><slot name="actions" /></footer>                 <!-- emplacement
nommé « actions » -->
  </div>
</template>
```

```
<!-- Parent : #titre est le raccourci de v-slot:titre -->
<Modale>
  <template #titre>Confirmation</template>

  <p>Voulez-vous continuer ?</p>              <!-- contenu hors <template> : va
dans le slot par défaut -->

  <template #actions>
    <button>Oui</button>
    <button>Non</button>
  </template>
</Modale>
<!-- → <div class="modale">
         <header>Confirmation</header>
         <main><p>Voulez-vous continuer ?</p></main>
         <footer><button>Oui</button><button>Non</button></footer>
       </div>
     Sans le bloc #titre, le header afficherait « Sans titre » (contenu par
défaut). -->
```

4.3 Slots à portée limitée (*scoped slots*)

L'enfant **fournit des données** au contenu injecté par le parent. Utile pour un composant de liste dont le **parent** décide de l'affichage de **chaque ligne**, alors que l'**enfant** possède la boucle et les données.

```vue
<!-- src/components/ListeGenerique.vue -->
<script setup>
defineProps({ elements: Array });
</script>

<template>
  <ul>
    <li v-for="(el, index) in elements" :key="index">
      <!-- On « expose » element et index au parent via les attributs du slot -->
      <slot :element="el" :index="index">{{ el }}</slot>
      <!-- Contenu par défaut : la valeur brute -->
    </li>
  </ul>
</template>
```

```vue
<!-- Parent -->
<script setup>
import { ref } from "vue";
import ListeGenerique from "./components/ListeGenerique.vue";

const personnes = ref([
  { nom: "Ali", ville: "Tunis" },
  { nom: "Ines", ville: "Sfax" }
]);
const villes = ref(["Djerba", "Gabès"]);
</script>

<template>
  <!-- Le parent récupère les données du slot par déstructuration : { element,
index } -->
  <ListeGenerique :elements="personnes">
    <template #default="{ element, index }">
      {{ index + 1 }}. <strong>{{ element.nom }}</strong> ({{ element.ville }})
    </template>
  </ListeGenerique>
  <!-- → • 1. Ali (Tunis)      (Ali en gras)
       → • 2. Ines (Sfax) -->

  <!-- Sans contenu de slot : l'affichage par défaut {{ el }} est utilisé -->
  <ListeGenerique :elements="villes" />
  <!-- → • Djerba
       → • Gabès -->
</template>
```

Le **même composant** `ListeGenerique` affiche des personnes avec une mise en forme riche, ou de simples villes : c'est ce qui le rend **réutilisable**.

4.4 Quand utiliser quoi ?

Besoin	Outil
Transmettre une donnée simple	prop
Prévenir le parent d'une action	événement (`emit`)
Laisser le parent fournir une portion d'interface	slot
Laisser le parent personnaliser l'affichage de données possédées par le composant	slot à portée limitée

---

5. Composition de composants et bonnes pratiques

5.1 Comment découper ?

Questions à se poser :

- Ce bloc est-il **répété** ? → composant réutilisable.
- A-t-il **une seule responsabilité** claire ? → composant.
- Le fichier dépasse-t-il **200–300 lignes** ou mélange-t-il plusieurs sujets ? → à découper.

5.2 Composants de présentation et composants « conteneurs »

- **Présentation** (*dumb*) : reçoit des props, émet des événements, aucun appel API, aucune dépendance globale. Très réutilisable et facile à tester.
- **Conteneur** (*smart*) : gère l'état, appelle les API, orchestre les composants de présentation.

5.3 Exemple : application de tâches en composants

```
src/
├─ App.vue                    ← conteneur : détient l'état, réagit aux
événements
└─ components/
   ├─ FormulaireTache.vue     ← émet « ajouter »
   ├─ ListeTaches.vue         ← props: taches ; relaie « supprimer », « basculer
»
   ├─ ItemTache.vue           ← (voir §3.1)
   └─ CompteurTaches.vue      ← props: total, restantes
```

```
<!-- src/components/FormulaireTache.vue -->
<script setup>
import { ref } from "vue";

const emit = defineEmits(["ajouter"]);
const texte = ref("");                 // état LOCAL au formulaire (le champ de
saisie)

function valider() {
  if (!texte.value.trim()) return;     // ignore un texte vide
  emit("ajouter", texte.value.trim()); // signale au parent : « voici une nouvelle
```

```
tâche »
  texte.value = "";                    // vide le champ
}
</script>

<template>
  <form @submit.prevent="valider">
    <input v-model="texte" placeholder="Nouvelle tâche">
    <button>Ajouter</button>
  </form>
</template>
```

```
<!-- src/components/ListeTaches.vue : relaie les événements d'ItemTache vers App -
->
<script setup>
import ItemTache from "./ItemTache.vue";

defineProps({ taches: Array });
const emit = defineEmits(["supprimer", "basculer"]);
</script>

<template>
  <ul>
    <!-- $event contient la valeur transmise par ItemTache (ici l'identifiant) -->
    <ItemTache v-for="t in taches" :key="t.id" :tache="t"
               @supprimer="emit('supprimer', $event)"
               @basculer="emit('basculer', $event)" />
  </ul>
  <p v-if="!taches.length">Aucune tâche.</p>
</template>
```

```
<!-- src/components/CompteurTaches.vue : pur affichage -->
<script setup>
defineProps({ total: Number, restantes: Number });
</script>

<template>
  <p>{{ restantes }} / {{ total }} tâche(s) restante(s)</p>
</template>
```

```
<!-- src/App.vue : le conteneur -->
<script setup>
import { ref, computed } from "vue";
import FormulaireTache from "./components/FormulaireTache.vue";
import ListeTaches from "./components/ListeTaches.vue";
import CompteurTaches from "./components/CompteurTaches.vue";
```

```
const taches = ref([]);                                    // seul endroit où
l'état existe
const restantes = computed(() => taches.value.filter(t => !t.fait).length);

const ajouter = (texte) => taches.value.push({ id: Date.now(), texte, fait: false
});
const supprimer = (id) => (taches.value = taches.value.filter(t => t.id !== id));
const basculer = (id) => {
  const t = taches.value.find(t => t.id === id);
  t.fait = !t.fait;
};
</script>

<template>
  <h1>Mes tâches</h1>
  <FormulaireTache @ajouter="ajouter" />
  <ListeTaches :taches="taches" @supprimer="supprimer" @basculer="basculer" />
  <CompteurTaches :total="taches.length" :restantes="restantes" />
</template>
```

Déroulement attendu :

Action	Chemin des données	Résultat à l'écran
Ouverture	—	« Aucune tâche. » et « 0 / 0 tâche(s) restante(s) »
Saisir « Réviser » + Entrée	`FormulaireTache` émet `ajouter("Réviser")` → `App.ajouter`	Une ligne « Réviser » ; « 1 / 1 tâche(s) restante(s) »
Cocher la case	`ItemTache` émet `basculer(id)` → `ListeTaches` relaie → `App.basculer`	Ligne barrée ; « 0 / 1 tâche(s) restante(s) »
Cliquer sur ×	`ItemTache` émet `supprimer(id)` → relayé → `App.supprimer`	La ligne disparaît ; « Aucune tâche. »

Schéma des échanges :

```
App (état : taches)
├─ FormulaireTache      ↑ ajouter(texte)
├─ ListeTaches          ↓ taches            ↑ supprimer(id), basculer(id)
│   └─ ItemTache        ↓ tache             ↑ supprimer(id), basculer(id)
└─ CompteurTaches       ↓ total, restantes
```

5.4 Bonnes pratiques

- **Une prop = une donnée simple** ; évitez de passer 15 props (regrouper dans un objet, ou revoir le découpage).
- Un composant ne modifie **jamais** ses props ; il émet des événements.
- Nommer les événements de manière cohérente (ce qui s'est passé).

- Style `scoped` pour éviter les fuites CSS.
- Un composant par fichier, nom en PascalCase composé de plusieurs mots.
- Pour partager des données avec des composants éloignés dans l'arbre, la transmission de props sur de nombreux niveaux (*prop drilling*) devient pénible : voir `provide`/`inject` (chapitre 7) et Pinia (chapitre 9).

5.5 Attributs hérités (*fallthrough*)

Les attributs ou écouteurs passés à un composant **sans être déclarés comme props** (`class`, `id`, `@click`…) sont automatiquement appliqués à l'**élément racine** du composant enfant.

```
<!-- src/components/BoutonBase.vue : aucune prop déclarée -->
<template>
  <button class="bouton"><slot /></button>
</template>
```

```
<!-- Parent -->
<BoutonBase class="grand" id="ok"
@click="console.log('clic')">Valider</BoutonBase>
<!-- → <button class="bouton grand" id="ok">Valider</button>
     (class est FUSIONNÉ avec « bouton » ; id est ajouté ; le clic déclenche
console.log) -->
```

Pour appliquer ces attributs à **un autre élément** que la racine :

```
<!-- src/components/ChampAvecLabel.vue -->
<script setup>
defineOptions({ inheritAttrs: false });    // désactive l'application automatique
sur la racine
</script>

<template>
  <label>
    <slot />
    <!-- $attrs contient tous les attributs reçus ; v-bind les applique ici -->
    <input v-bind="$attrs">
  </label>
</template>
```

```
<ChampAvecLabel placeholder="Votre nom" maxlength="20">Nom :</ChampAvecLabel>
<!-- → <label>Nom : <input placeholder="Votre nom" maxlength="20"></label>
     (les attributs vont sur l'<input>, pas sur le <label> racine) -->
```

Résumé

- Un composant est un fichier `.vue` ; un composant **importé** est directement utilisable avec `<script setup>`.
- **Props** (parent → enfant, lecture seule), **événements** (`emit`, enfant → parent), **slots** (contenu injecté).
- `defineProps`, `defineEmits`, `defineModel` : macros du compilateur, sans import.
- Une prop n'est jamais modifiée par l'enfant : état local initialisé par la prop, valeur dérivée (`computed`), ou événement vers le parent.
- Découper selon la **responsabilité** et la **réutilisation** ; séparer présentation (props/événements) et logique (conteneur).